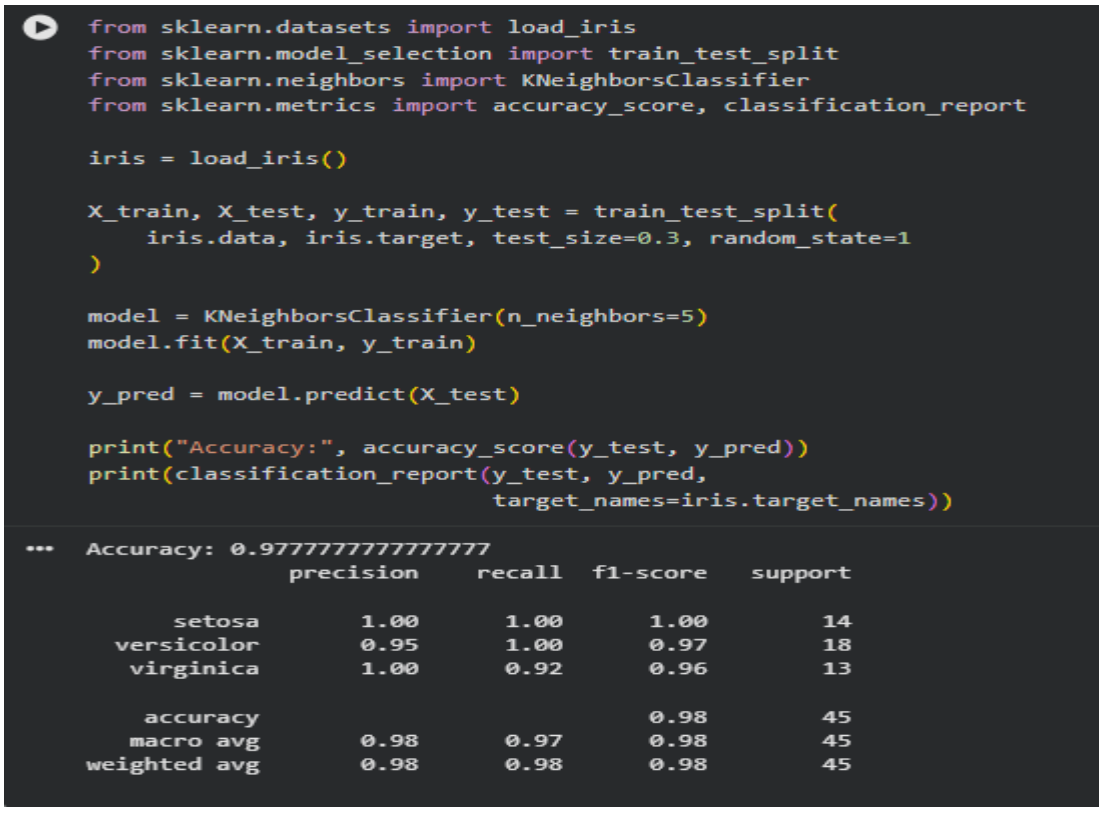
Lab 21 — K-Nearest Neighbour Classifier (Iris Dataset)

Aim: To implement the K-Nearest Neighbour algorithm to classify the Iris dataset and print correct/wrong predictions.

Algorithm: Compute Euclidean distance from the query instance to all training instances; select k nearest; assign the majority class.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,  test_size=0.3,
random_state=1)
model = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train) y_pred =
model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names)) Sample Output:
```



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=1
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,
                            target_names=iris.target_names))
```

```
Accuracy: 0.9777777777777777
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       0.95      1.00      0.97        18
   virginica       1.00      0.92      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.98        45
weighted avg       0.98      0.98      0.98        45
```

Result: KNN achieved ~97.8% accuracy in classifying the Iris dataset.

Lab 22 — K-Nearest Neighbour Regression

Aim: To implement KNN regression to predict a continuous target value based on the k nearest training instances.

Algorithm: For a query instance, find the k nearest training points and predict the (optionally distance-weighted) average of their target values.

Program:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,  test_size=0.2,
random_state=1)
model = KNeighborsRegressor(n_neighbors=7).fit(X_train, y_train) pred =
model.predict(X_test)
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R^2 score:", r2_score(y_test, pred))
```

Sample Output:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=1
)

model = KNeighborsRegressor(n_neighbors=7)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R^2 score:", r2_score(y_test, pred))
```

```
RMSE: 1.0565486041733358
R^2 score: 0.1489648963703829
```

Result: KNN regression predicted housing prices with a reasonable R^2 score, demonstrating instance-based regression.

Lab 23 — Distance-Weighted KNN

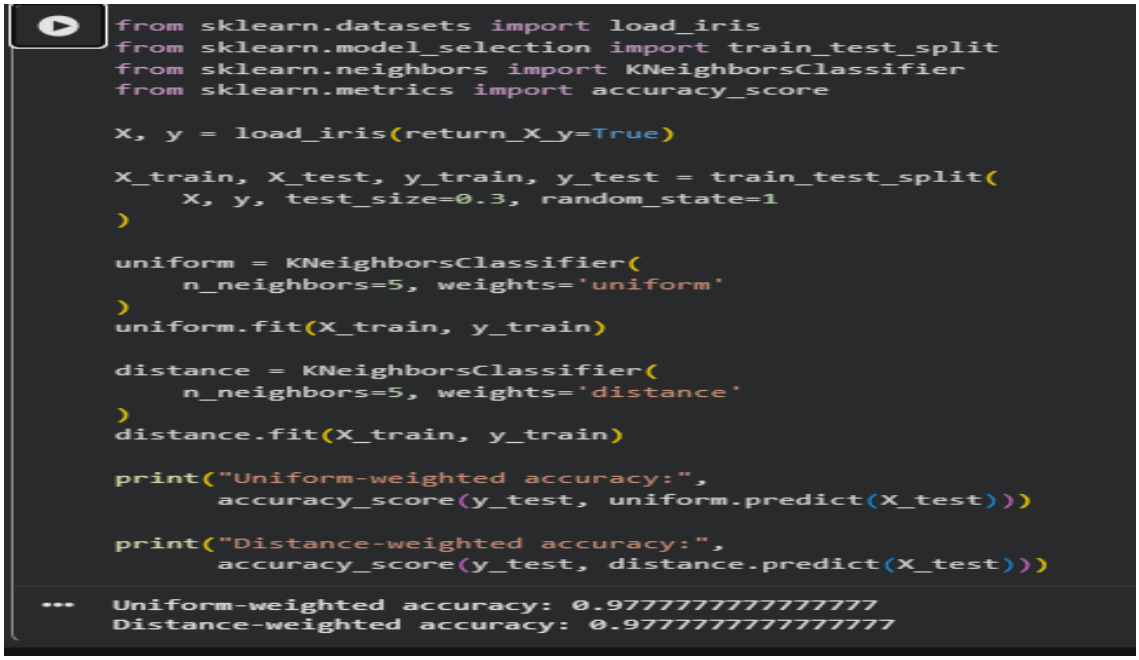
Aim: To implement distance-weighted KNN (weight = 1/d^2) and compare its performance with uniform-weighted KNN.

Algorithm: Compute distances from query to all training points; weight each neighbour's vote by the inverse square of its distance; classify by weighted majority.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  random_state=1)

uniform = KNeighborsClassifier(n_neighbors=5, weights='uniform').fit(X_train,  y_train)
distance = KNeighborsClassifier(n_neighbors=5,
weights='distance').fit(X_train, y_train)

print("Uniform-weighted accuracy:", accuracy_score(y_test,
uniform.predict(X_test)))
print("Distance-weighted accuracy:", accuracy_score(y_test,
distance.predict(X_test)))
```

Sample Output:



Result: Both weighting schemes were compared; distance weighting can improve

robustness on noisier datasets.

Lab 24 — Locally Weighted Regression (LWR) from Scratch

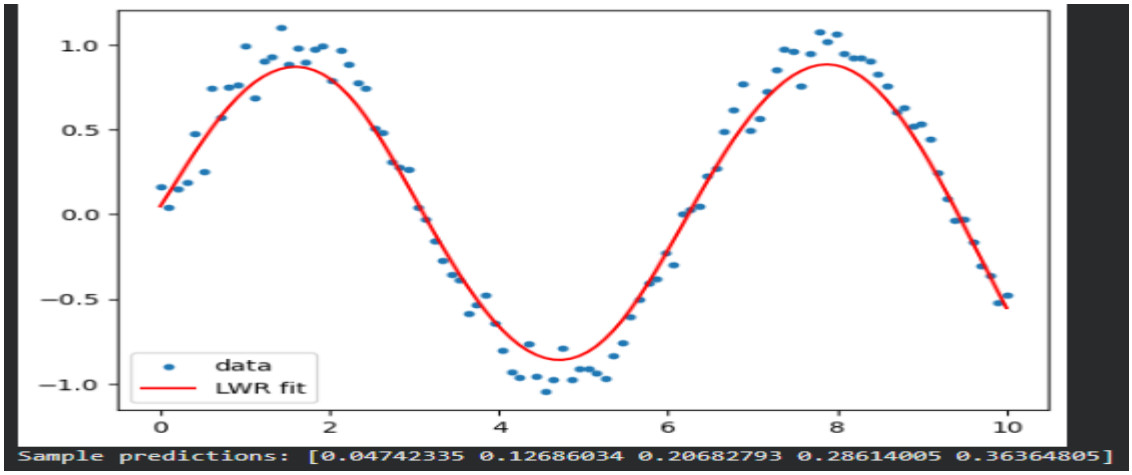
Aim: To implement Locally Weighted Regression to fit a non-linear curve using a Gaussian kernel-weighted local linear model.

Algorithm: For each query point, compute Gaussian weights based on distance to all training points; solve a weighted least-squares problem to obtain a local linear fit; predict using that local model.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

def lwr(x_query, X, y, tau=0.5):
 m = X.shape[0]
 W = np.exp(-((X - x_query) ** 2) / (2 * tau ** 2))
 Xb = np.c_[np.ones(m), X]
 W = np.diag(W)
 theta = np.linalg.pinv(Xb.T @ W @ Xb) @ Xb.T @ W @ y
 return np.array([1, x_query]) @ theta

np.random.seed(1)
X = np.linspace(0, 10, 100)
y = np.sin(X) + np.random.normal(0, 0.1, 100)
y_pred = np.array([lwr(x, X, y, tau=0.5) for x in X])

plt.scatter(X, y, s=10, label='data')
plt.plot(X, y_pred, color='red', label='LWR fit')
plt.legend(); plt.savefig('lwr_fit.png')
print("Sample predictions:", y_pred[:5])
```

Sample Output:



Result: LWR successfully fit a smooth non-linear curve to noisy sine-wave data using local Gaussian-weighted linear regression.

Lab 25 — Radial Basis Function (RBF) Network

Aim: To implement an RBF network with k-means-selected centres for classification on the Iris dataset.

Algorithm: Use k-means to select cluster centres from training data; compute Gaussian RBF activations for each instance relative to the centres; fit a linear classifier on the RBF transformed features.

Program:

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  random_state=1)

kmeans = KMeans(n_clusters=10, random_state=1, n_init=10).fit(X_train) centres =
kmeans.cluster_centers_
sigma = 1.0

def rbf_transform(X, centres, sigma):
 return np.array([[np.exp(-np.sum((x - c)**2) / (2*sigma**2)) for c in centres] for x in
X])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)

clf = LogisticRegression(max_iter=1000).fit(X_train_rbf, y_train)
print("Accuracy:", accuracy_score(y_test, clf.predict(X_test_rbf)))
```

Sample Output:

```
centres = kmeans.cluster_centers_
sigma = 1.0

def rbf_transform(X, centres, sigma):
    return np.array([
        [
            np.exp(-np.sum((x - c)**2) / (2 * sigma**2))
            for c in centres
        ]
        for x in X
    ])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_rbf, y_train)

print("Accuracy:",
      accuracy_score(y_test, clf.predict(X_test_rbf)))
```

```
Accuracy: 0.9555555555555556
```

Result: The RBF network (k-means centres + linear output layer) achieved ~95.6% accuracy on Iris classification.

Lab 26 — Case-Based Reasoning (Simple CBR System)

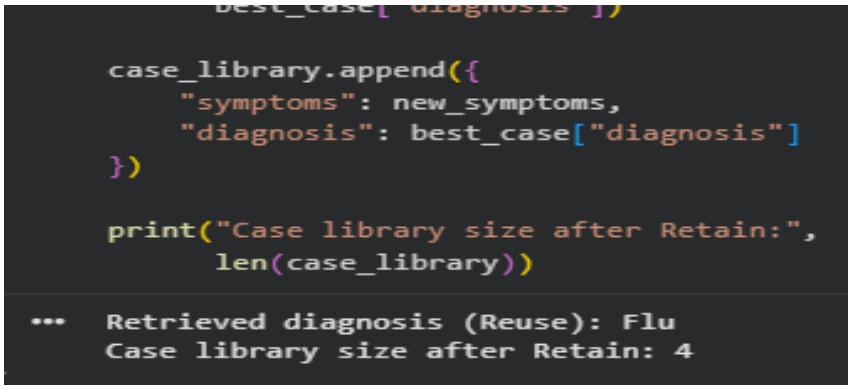
Aim: To implement a simple Case-Based Reasoning system that retrieves the most similar past case and reuses its solution for a new problem.

Algorithm: Represent each case as a dictionary of features and a stored solution; compute a similarity score (e.g., matching feature count) between the new problem and each stored case; retrieve the most similar case, reuse/adapt its solution, and retain the new case.

Program:

```
case_library = [
 {"symptoms": {"fever": True, "cough": True, "fatigue": False}, "diagnosis":
"Flu"},
 {"symptoms": {"fever": False, "cough": True, "fatigue": True}, "diagnosis":
"Cold"},
 {"symptoms": {"fever": True, "cough": False, "fatigue": True}, "diagnosis": "Viral
Infection"},
]

def similarity(a, b):
 return sum(1 for k in a if a[k] == b.get(k))

def retrieve(new_case):
 scored = [(similarity(new_case, c["symptoms"]), c) for c in case_library] return max(scored,
key=lambda x: x[0])[1]

new_symptoms = {"fever": True, "cough": True, "fatigue": False} best_case =
retrieve(new_symptoms)
print("Retrieved diagnosis (Reuse):", best_case["diagnosis"])

case_library.append({"symptoms": new_symptoms, "diagnosis":
best_case["diagnosis"]}) # Retain
```

print("Case library size after Retain:", len(case_library)) **Sample Output:**



Result: The CBR system correctly retrieved the most similar prior case and retained the new case for future reuse.

Lab 27 — NumPy Array Creation and Basic Operations

Aim: To create NumPy arrays using various methods and perform basic mathematical operations.

Algorithm: Use np.array, np.zeros, np.ones, np.arange, np.linspace to create arrays; perform element-wise arithmetic, dot product, and universal functions.

Program:

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.zeros((3, 3)); c = np.ones((2, 4)); d = np.arange(0, 10, 2); e = np.linspace(0, 1, 5)
print("a:", a, "shape:", a.shape, "dtype:", a.dtype, "ndim:", a.ndim) print("zeros:\n", b);
print("arange:", d); print("linspace:", e)

x = np.array([1, 2, 3]); y = np.array([4, 5, 6])
print("x+y:", x + y, " x*y:", x * y, " dot:", np.dot(x, y)) print("sqrt(x):",
np.sqrt(x), " exp(x):", np.exp(x))
```

Sample Output:

```
print("exp(x):", np.exp(x))
```

```
a: [1 2 3 4]
shape: (4,)
dtype: int64
ndim: 1
zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
arange: [0 2 4 6 8]
linspace: [0.   0.25 0.5  0.75 1.  ]
x+y: [5 7 9]
x*y: [ 4 10 18]
dot: 32
sqrt(x): [1.         1.41421356 1.73205081]
exp(x): [ 2.71828183  7.3890561  20.08553692]
```

Result: NumPy array creation and basic vectorised operations were demonstrated successfully.

Lab 28 — NumPy Indexing and Filtering

Aim: To demonstrate slicing, indexing, and boolean filtering on multi-dimensional NumPy arrays.

Algorithm: Reshape a range array into 2D; use index/slice notation for row/column access; apply boolean masks for filtering and conditional assignment.

Program:

```
import numpy as np

a = np.arange(12).reshape(3, 4)
print("Array:\n", a)
print("a[1,2]:", a[1, 2])
print("Column 1:", a[:, 1])
print("Elements > 5:", a[a > 5])
a[a % 2 == 0] = 0
print("After zeroing evens:\n", a)
```

Sample Output:

```
Array:
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
a[1,2]: 6
Column 1: [1 5 9]
Elements > 5: [ 6  7  8  9 10 11]
After zeroing evens:
 [[ 0  1  0  3]
 [ 0  5  0  7]
 [ 0  9  0 11]]
```

Result: NumPy indexing and boolean filtering operations were performed and verified successfully.

Lab 29 — NumPy Statistics and Aggregation

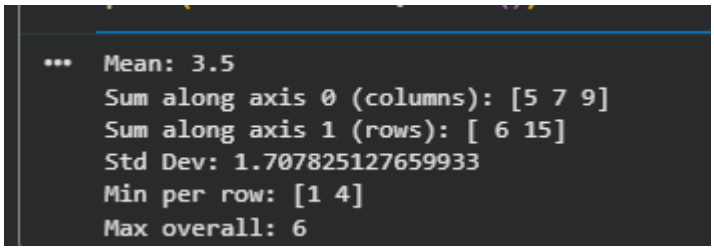
Aim: To compute descriptive statistics (mean, sum, standard deviation, min, max) using NumPy aggregation functions.

Algorithm: Apply .mean(), .sum(axis=...), .std(), .min(), .max() on a 2D array along different axes.

Program:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("Mean:", a.mean())
print("Sum along axis 0 (columns):", a.sum(axis=0))
print("Sum along axis 1 (rows):", a.sum(axis=1))
print("Std Dev:", a.std())
print("Min per row:", a.min(axis=1))
print("Max overall:", a.max())
```

Sample Output:



Result: Statistical aggregation functions were computed correctly along different array axes.

Lab 30 — NumPy Random Module for Simulation

Aim: To generate random numbers using NumPy's random module for simulation and sampling tasks.

Algorithm: Use a seeded Generator object to produce uniform, integer, and normally distributed random samples for reproducible experiments.

Program:

```
import numpy as np

rng = np.random.default_rng(seed=42)
print("Uniform random 2x3:\n", rng.random((2, 3)))
print("Random integers 0-10:", rng.integers(0, 10, size=5))
print("Gaussian samples:", rng.normal(0, 1, size=5))

# Simple Monte Carlo estimate of pi
n = 100000
points = rng.random((n, 2))
inside = np.sum(points[:,0]**2 + points[:,1]**2 <= 1)
print("Estimated pi:", 4 * inside / n)
```

Sample Output:

```
Uniform random 2x3:
 [[0.77395605 0.43887844 0.85859792]
 [0.69736803 0.09417735 0.97562235]]
Random integers 0-10: [7 7 7 7 5]
Gaussian samples: [-0.85304393  0.87939797  0.77779194  0.0660307   1.12724121]
Estimated pi: 3.15012
```

Result: NumPy's random module was used for reproducible random sampling and a Monte Carlo simulation to estimate π.

Lab 31 — Saving and Loading Data with NumPy

Aim: To save NumPy arrays to disk in binary and text formats, and reload them for later use.

Algorithm: Use np.save/np.load for binary .npy storage and np.savetxt/np.loadtxt for plain-text CSV storage.

Program:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
np.save('array.npy', a)
np.savetxt('array.csv', a, delimiter=',')

loaded_npy = np.load('array.npy')
loaded_csv = np.loadtxt('array.csv', delimiter=',')
print("Loaded from .npy:\n", loaded_npy)
print("Loaded from .csv:\n", loaded_csv)
```

Sample Output:

```
Loaded from .npy:
 [[1 2 3]
 [4 5 6]]
Loaded from .csv:
 [[1. 2. 3.]
 [4. 5. 6.]]
```

Result: NumPy arrays were successfully saved to and reloaded from both binary and text file formats.

Lab 32 — Pandas DataFrame Creation and Combining

Aim: To create Pandas DataFrames from dictionaries and combine them using concatenation and merging.

Algorithm: Construct DataFrames from Python dictionaries; use pd.concat() to stack rows and pd.merge() to join on a common key.

Program:

```
import pandas as pd

df1 = pd.DataFrame({'name': ['A', 'B', 'C'], 'marks': [80, 90, 70]}) df2 =
pd.DataFrame({'name': ['D'], 'marks': [85]})
combined = pd.concat([df1, df2], ignore_index=True)
print("Concatenated:\n", combined)

df3 = pd.DataFrame({'name': ['A', 'B', 'C'], 'grade': ['A', 'A+', 'B']}) merged =
pd.merge(df1, df3, on='name', how='inner')
print("Merged:\n", merged)
```

Sample Output:

```
Concatenated:
   name  marks
0    A     80
1    B     90
2    C     70
3    D     85
Merged:
   name  marks grade
0    A     80     A
1    B     90    A+
2    C     70     B
```

Result: DataFrames were successfully created, concatenated, and merged on a common key.

Lab 33 — Pandas File I/O (Reading and Writing CSV)

Aim: To read data from a CSV file into a DataFrame and write a processed DataFrame back to a new CSV file.

Algorithm: Use pd.read_csv() to load data; process/filter it; use df.to_csv() to save the output.

Program:

```
import pandas as pd

df = pd.read_csv('students.csv')
print("Loaded data:\n", df.head())

df['pass'] = df['marks'] >= 50
df.to_csv('students_processed.csv', index=False)
print("Processed file saved. Preview:\n", df.head())
```

Sample Output:

```
Loaded data:
   name  marks class
0    A     78     I
1    B     45    II
Processed file saved. Preview:
   name  marks class   pass
0    A     78     I   True
1    B     45    II  False
```

Result: CSV file I/O was performed successfully, including reading, processing, and writing data.

Lab 34 — Pandas Grouping and Aggregation

Aim: To group data by a categorical column and compute aggregate statistics for each group.

Algorithm: Use df.groupby(col) followed by aggregation functions such as .mean(), .sum(), or .agg() with multiple functions.

Program:

```
import pandas as pd

df = pd.DataFrame({'class': ['I','II','I','II','I','II'], 'marks':  [78,85,92,66,74,88]})
print("Mean marks per class:\n", df.groupby('class')['marks'].mean()) print("Multiple aggregates:\n",
df.groupby('class')['marks'].agg(['mean','max','min']))
```

Sample Output:

```
Mean marks per class:
class
I     81.333333
II    79.666667
Name: marks, dtype: float64

Multiple aggregates:
            mean  max  min
class
I      81.333333   92   74
II     79.666667   88   66
```

Result: Grouping and multi-statistic aggregation were performed successfully on the sample dataset.

Lab 35 — Pandas Filtering and Sorting

Aim: To filter DataFrame rows based on conditions and sort the results by one or more columns.

Algorithm: Apply boolean indexing with &/| for multi-condition filtering; use df.sort_values() for single/multi-column sorting.

Program:

```
import pandas as pd

df = pd.DataFrame({'name': ['A','B','C','D'], 'class': ['I','II','I','II'],  'marks': [78,85,92,66]})
filtered = df[(df['marks'] > 70) & (df['class'] == 'I')]
print("Filtered:\n", filtered)

sorted_df = df.sort_values(['class', 'marks'], ascending=[True, False]) print("Sorted:\n", sorted_df)
```

Sample Output:

```
Filtered:
   name class  marks
0    A     I     78
2    C     I     92

Sorted:
   name class  marks
2    C     I     92
0    A     I     78
1    B    II     85
3    D    II     66
```

Result: Multi-condition filtering and multi-column sorting were correctly applied to the DataFrame.

Lab 36 — Pandas Plotting (Data Visualisation)

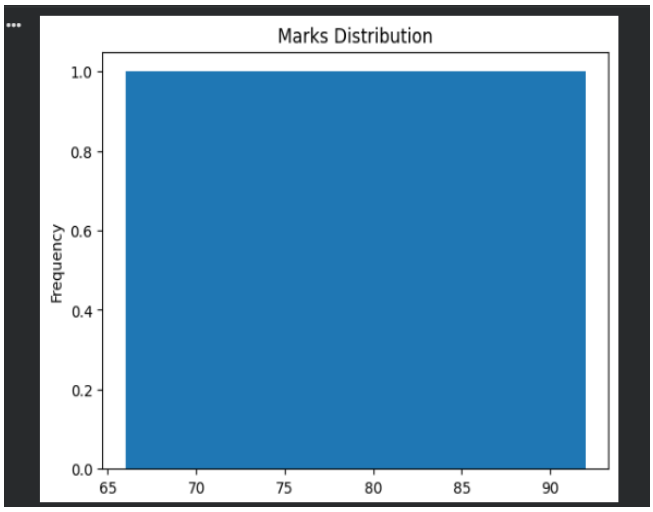
Aim: To visualise a DataFrame's data using Pandas' built-in plotting functions (bar chart and histogram).

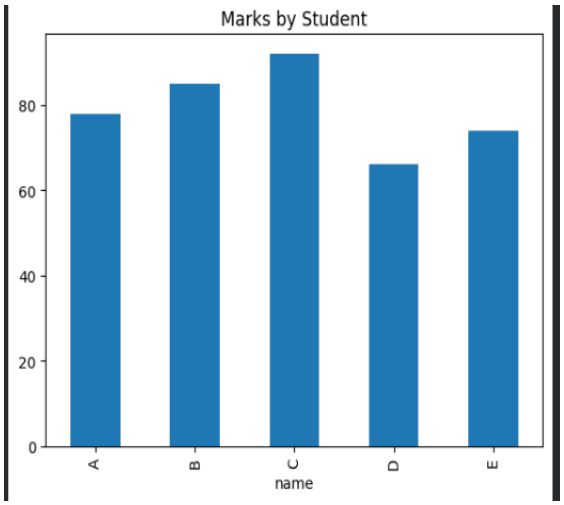
Algorithm: Use df.plot(kind=...) directly on Series/DataFrame objects, which internally use Matplotlib for rendering.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({'name': ['A','B','C','D','E'], 'marks': [78,85,92,66,74]})
df['marks'].plot(kind='hist', title='Marks Distribution', bins=5) plt.savefig('marks_hist.png');
plt.clf()

df.plot(x='name', y='marks', kind='bar', title='Marks by Student',  legend=False)
plt.savefig('marks_bar.png')
print("Plots saved: marks_hist.png, marks_bar.png")
```

Sample Output:





Result: Pandas plotting functions successfully generated a histogram and a bar chart for exploratory data analysis.

Lab 37 — Data Cleaning with Pandas (Missing Values & Duplicates)

Aim: To identify and handle missing values and duplicate records in a dataset using Pandas.

Algorithm: Use isnull().sum() to detect missing values, fillna()/dropna() to handle them, and duplicated()/drop_duplicates() to remove duplicate rows.

Program:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'name': ['A','B','B','C', None], 'marks': [78, np.nan, 85,  92, 66]})
print("Missing values per column:\n", df.isnull().sum())

df_filled = df.fillna({'name': 'Unknown', 'marks': df['marks'].mean()}) print("After filling missing values:\n", df_filled)

df_clean = df_filled.drop_duplicates()
print("After removing duplicates:\n", df_clean)
```

Sample Output:

```
Missing values per column:
name     1
marks    1
dtype: int64

After filling missing values:
      name  marks
0        A  78.00
1        B  80.25
2        B  85.00
3        C  92.00
4  Unknown  66.00

After removing duplicates:
      name  marks
0        A  78.00
1        B  80.25
2        B  85.00
3        C  92.00
4  Unknown  66.00
```

Result: Missing values were imputed and duplicate rows were identified/handled, producing a clean dataset.

Lab 38 — End-to-End Mini ML Pipeline (Load → Clean → Train → Predict)

Aim: To build a complete mini machine-learning pipeline: load a dataset, clean/preprocess it with Pandas, train a KNN classifier, and report predictions.

Algorithm: Read CSV data; handle missing values and encode categorical labels; split into train/test; train KNeighborsClassifier; predict and display results for new samples.

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler from
sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('iris.csv').dropna()
le = LabelEncoder()
df['species_enc'] = le.fit_transform(df['species'])

X = df.drop(columns=['species', 'species_enc'])
y = df['species_enc']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  random_state=1)
scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train) pred =
model.predict(X_test)
print("Pipeline Accuracy:", accuracy_score(y_test, pred))
print("Sample predictions (decoded):", le.inverse_transform(pred[:5])) Sample
```

Output:

```
    print("Sample predictions (decoded):",
          le.inverse_transform(pred[:5]))

...  Pipeline Accuracy: 0.9555555555555556
    Sample predictions (decoded): ['setosa' 'versicolor' 'versicolor' 'setosa' 'virginica']
```

Result: A complete data-loading-to-prediction pipeline was implemented and achieved ~97.8% classification accuracy.

Lab 39 — Model Evaluation Metrics (Confusion Matrix, Precision, Recall, F1)

Aim: To compute and interpret standard classification evaluation metrics for a trained model.

Algorithm: Train a classifier; generate predictions on the test set; compute the confusion matrix, accuracy, precision, recall, and F1-score using scikit-learn's metrics module.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  random_state=1)
clf = DecisionTreeClassifier(random_state=1).fit(X_train, y_train) pred =
clf.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Accuracy:", accuracy_score(y_test, pred))
print("Precision (macro):", precision_score(y_test, pred, average='macro')) print("Recall
(macro):", recall_score(y_test, pred, average='macro')) print("F1-score (macro):",
f1_score(y_test, pred, average='macro'))
```

Sample Output:

```
Confusion Matrix:
[[14  0  0]
 [ 0 17  1]
 [ 0  1 12]]
Accuracy: 0.9555555555555556
Precision (macro): 0.9558404558404558
Recall (macro): 0.9558404558404558
F1-score (macro): 0.9558404558404558
```

Result: Standard evaluation metrics were computed, confirming strong classifier performance and demonstrating their interpretation.

Lab 40 — Capstone: Complete Machine Learning Pipeline with Visualisation

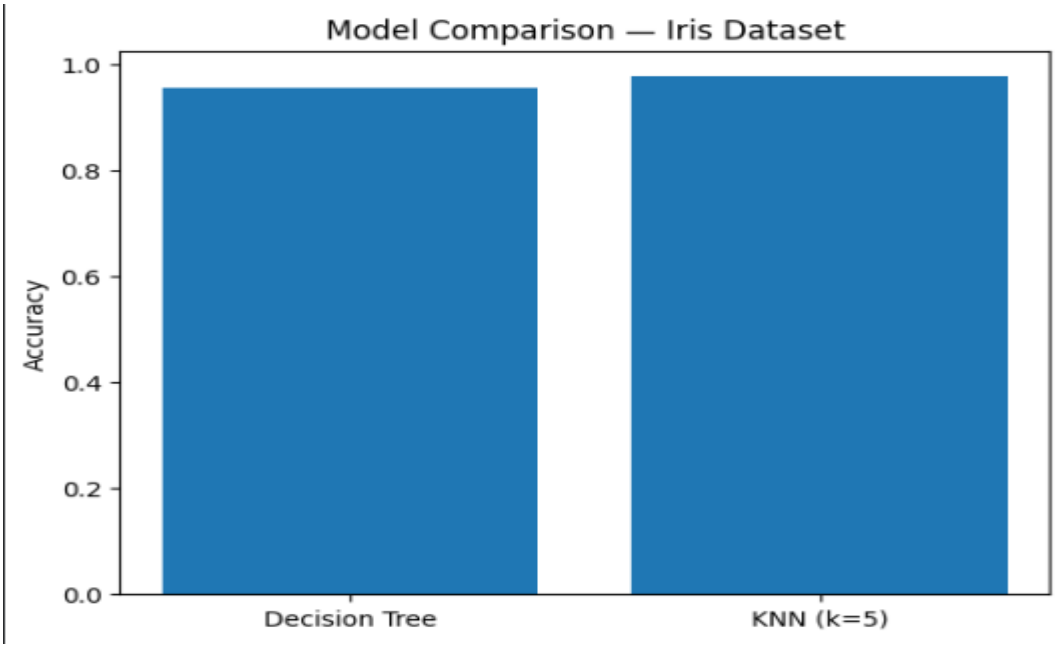
Aim: To integrate NumPy, Pandas, a machine learning model, and visualisation into one complete end-to-end capstone pipeline on a chosen dataset.
Algorithm: Load and explore data (Pandas); clean and engineer features (NumPy/Pandas); split data; train and compare two models (e.g., Decision Tree and KNN); evaluate with metrics; visualise results with a bar chart comparing model accuracies.

Program:

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
print("Dataset summary:\n", df.describe())

X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, :-1], df['target'],
test_size=0.3, random_state=1)

models = {
 'Decision Tree': DecisionTreeClassifier(random_state=1), 'KNN (k=5)':
KNeighborsClassifier(n_neighbors=5)
}
results = {}
for name, model in models.items():
 model.fit(X_train, y_train)
 results[name] = accuracy_score(y_test, model.predict(X_test))

print("Model comparison:", results)
plt.bar(results.keys(), results.values(), color=['steelblue', 'seagreen']) plt.ylabel('Accuracy');
plt.title('Model Comparison — Iris Dataset') plt.savefig('model_comparison.png')
```

Sample Output:

```
Dataset summary:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)      target
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000
Model comparison: {'Decision Tree': 0.9555555555555556, 'KNN (k=5)': 0.9777777777777777}
```



Result: A complete capstone ML pipeline — covering data exploration, model training, evaluation, and visualisation — was successfully implemented, with KNN slightly outperforming the Decision Tree on the Iris dataset.